

Московский государственный университет имени М.В. Ломоносова
Факультет вычислительной математики и кибернетики

**Отчет о выполнении
практического задания №4 по курсу
”Суперкомпьютерное моделирование и
технологии”**

ВАРИАНТ 1

Выполнил:
Дроздов Никита Александрович
группа 622

Москва
2025

1 Математическая постановка дифференциальной задачи

В трёхмерной замкнутой области

$$\Omega = [0 \leq x \leq L_x] \times [0 \leq y \leq L_y] \times [0 \leq z \leq L_z]$$

для $0 < t \leq T$ требуется найти решение $u(x, y, z, t)$ уравнения в частных производных

$$\frac{\partial^2 u}{\partial t^2} = a^2 \Delta u, \quad (1)$$

с начальными условиями

$$u|_{t=0} = \varphi(x, y, z), \quad (2)$$

$$\left. \frac{\partial u}{\partial t} \right|_{t=0} = 0, \quad (3)$$

и граничными условиями варианта 1:

$$\begin{aligned} u(0, y, z, t) &= 0, & u(L_x, y, z, t) &= 0, \\ u(x, 0, z, t) &= 0, & u(x, L_y, z, t) &= 0, \\ u(x, y, 0, t) &= 0, & u(x, y, L_z, t) &= 0. \end{aligned} \quad (4)$$

2 Численный метод решения задачи

Для численного решения задачи введём на Ω сетку $\omega_{h\tau} = \bar{\omega}_h \times \omega_\tau$, где

$$\bar{\omega}_h = \{(x_i = ih_x, y_j = jh_y, z_k = kh_z), i, j, k = 0, 1, \dots, N\},$$

$$h_x N = L_x, \quad h_y N = L_y, \quad h_z N = L_z,$$

$$\omega_\tau = \{t_n = n\tau, n = 0, 1, \dots, K, \tau K = T\}.$$

Аппроксимируем уравнение (1) явной разностной схемой:

$$\frac{u_{ijk}^{n+1} - 2u_{ijk}^n + u_{ijk}^{n-1}}{\tau^2} = a^2 \Delta_h u_{ijk}^n, \quad (x_i, y_j, z_k) \in \omega_h, \quad n = 1, 2, \dots, K-1,$$

где Δ_h — семиточечный разностный аналог оператора Лапласа:

$$\Delta_h u_{ijk}^n = \frac{u_{i-1,j,k}^n - 2u_{i,j,k}^n + u_{i+1,j,k}^n}{h_x^2} + \frac{u_{i,j-1,k}^n - 2u_{i,j,k}^n + u_{i,j+1,k}^n}{h_y^2} + \frac{u_{i,j,k-1}^n - 2u_{i,j,k}^n + u_{i,j,k+1}^n}{h_z^2}.$$

Начальные условия аппроксимируем следующим образом:

$$u_{ijk}^0 = \varphi(x_i, y_j, z_k), \quad (x_i, y_j, z_k) \in \omega_h, \quad (5)$$

$$u_{ijk}^1 = u_{ijk}^0 + a^2 \frac{\tau^2}{2} \Delta_h \varphi(x_i, y_j, z_k), \quad (x_i, y_j, z_k) \in \omega_h. \quad (6)$$

Граничные условия для варианта 1 задаются следующим образом:

$$u_{0jk}^{n+1} = u_{Njk}^{n+1} = 0, \quad u_{i0k}^{n+1} = u_{iNk}^{n+1} = 0, \quad u_{ij0}^{n+1} = u_{ijN}^{n+1} = 0$$

для $i, j, k = 0, 1, \dots, N$. Константа a^2 в варианте 1 принимает следующее значение:

$$a^2 = \frac{L_x^2 L_y^2 L_z^2}{L_x^2 L_y^2 + L_y^2 L_z^2 + L_x^2 L_z^2}.$$

Аналитическое решение для варианта 1 задается следующим выражением:

$$u_{\text{analytical}} = \sin\left(\frac{\pi}{L_x}x\right) \cdot \sin\left(\frac{\pi}{L_y}y\right) \cdot \sin\left(\frac{\pi}{L_z}z\right) \cdot \cos(\pi t).$$

Начальное условие для варианта 1:

$$\varphi(x, y, z) = u_{\text{analytical}}(x, y, z, 0) = \sin\left(\frac{\pi}{L_x}x\right) \cdot \sin\left(\frac{\pi}{L_y}y\right) \cdot \sin\left(\frac{\pi}{L_z}z\right).$$

3 Программная реализация

Выполненное практическое задание №4 частично основывается на трех предыдущих заданиях. В рамках задания 1 требовалось реализовать параллельный алгоритм численного решения дифференциальной задачи (1) – (4) с использованием директив OpenMP. В рамках задания 2 требовалось распараллелить задачу с использованием стандарта MPI. В рамках задания 3 нужно было написать параллельную версию программы с использованием как стандарта MPI, так и директив OpenMP. Программный код всех четырех практических заданий доступен по ссылке: https://github.com/anac0der/hpc2025_cuda.

Рассмотрим основные этапы работы алгоритма, реализованного в практическом задании №4:

1. Обрабатываются аргументы командной строки – число узлов сетки N , количество шагов по времени K и величина одного шага τ ;
2. Инициализируется группа процессов с нужной топологией при помощи *MPI_Cart_create*, *MPI_Cart_shift*;
3. Происходит разбиение пространственной области на блоки в зависимости от числа процессов;
4. Инициализируются решения u^0 и u^1 . Для этого каждый процесс вызывает внутри себя CUDA-ядро для инициализации в соответствующем домене;
5. Далее в цикле по n вычисляются значения u^{n+1} по u^n и u^{n-1} . На каждой итерации перед вычислением решения производится обмен граничными значениями между процессами. Для этого реализована отдельная функция, использующая внутри себя *MPI_Sendrecv*. Обмен производится на хосте, для пересылки значений используется функция *cudaMemcpy*. После обмена каждый процесс вызывает CUDA-ядро для вычисления решения. В конце каждой итерации выводится значение погрешности численного метода с редукцией между процессами при помощи *MPI_Reduce*. Редукция ошибки внутри каждого домена производится на GPU с использованием библиотеки *thrust*;
6. В конце также выводится общее время работы программы, для его подсчета используется утилита *MPI_Wtime*.

Проверка корректности реализации осуществлялась следующим образом:

- Для каждой параллельной реализации полученная погрешность на каждой итерации сравнивалась с соответствующей ошибкой в последовательном алгоритме. Эти цифры получились одинаковыми для всех версий программы;
- Для распараллеливания с использованием MPI и CUDA после каждого изменения в программе проводился отладочный запуск, подтверждающий, что данное изменение не нарушило логику программы (вывод погрешностей на всех шагах и отладочных данных для каждого процесса). Для CUDA версии дополнительно проверялась загрузка видеокарт в момент исполнения программы.

4 Результаты расчетов

Все расчеты были выполнены на вычислительной машине IBM Polus. Рассматривалась пространственная область с $L_x = L_y = L_z = 1$ и сетка размера $N = 256$. Во всех запусках программы выполнялось 50 шагов по времени. Величина шага τ равнялась 0.002.

Таблица 1: Результаты расчетов для разных версий параллельной реализации численного метода.

Число MPI процессов	Число нитей	Число GPU	Время решения t , сек	Ускорение S	Погрешность ψ
1	—	—	122.561	1.000	4.94e-7
20	—	—	7.753	15.808	4.94e-7
20	8	—	3.387	36.181	4.94e-7
1	—	1	3.103	39.506	4.94e-7
2	—	2	1.756	69.786	4.94e-7

В таблице 1 представлены основные результаты расчетов для данного задания. В таблицах 3, 4, 5 приведены дополнительные расчеты для конкретных типов параллельных реализаций. Характер ускорений в таблице 1 можно объяснить следующим образом:

- При распараллеливании только с помощью MPI достигается хорошая пропорциональность ускорения относительно количества выделенных процессов (15.8 против 20 процессов), однако с ростом числа процессов соответствующий прирост ускорения может замедлиться из-за накладных расходов (см. таблицу 4);
- При дополнительном распараллеливании MPI программы с помощью OpenMP удалось получить прирост еще почти в 2.5 раза по скорости. Большой прирост в данной постановке получить сложно из-за большого числа MPI процессов и связанных с ними расходов (похожий эффект можно наблюдать в таблице 3);
- Распараллеливание программы с помощью только 1 GPU показало ускорение почти в 40 раз, что приблизительно равно ускорению на 160 потоках для MPI+OpenMP программы. Программа, запущенная на 2 GPU, показывает ускорение почти в 70 раз, что является лучшим результатом среди всех параллельных реализаций, представленных в данном отчете.

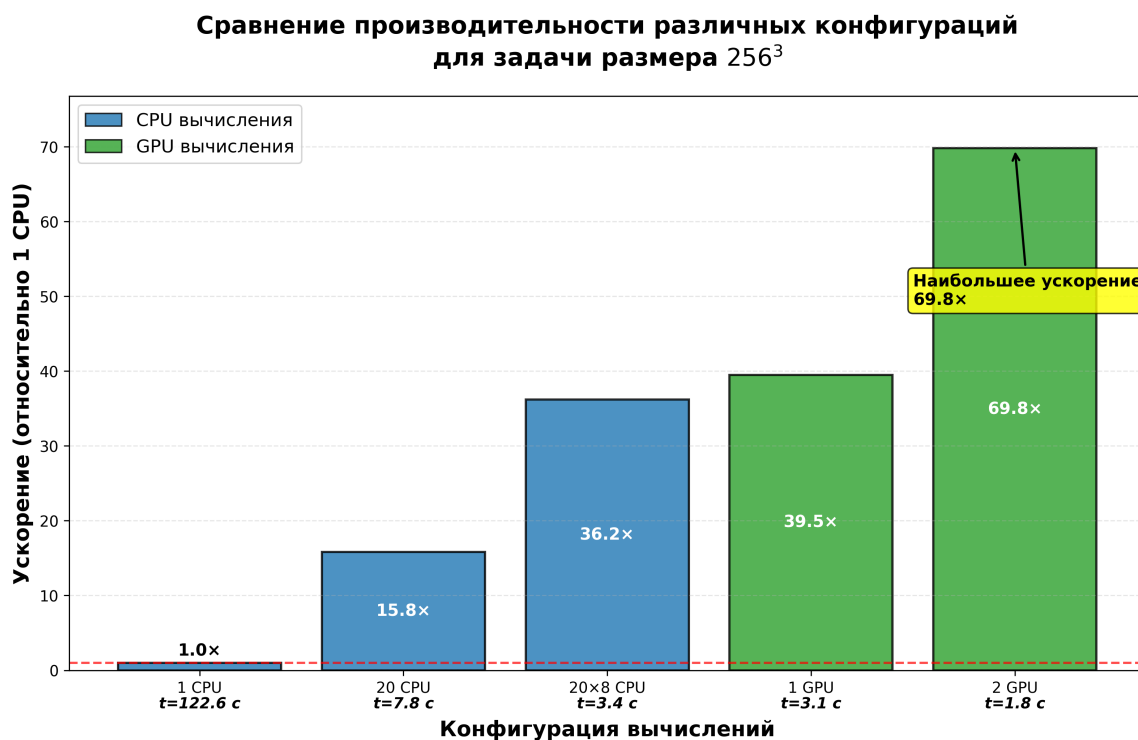


Рис. 1: Графики ускорений различных параллельных реализаций численного метода относительно последовательной версии программы.

На Рис. 1 приведено визуальное сравнение ускорений по данным из таблицы 1 для MPI+CUDA версии алгоритма. На графике видны все закономерности, описанные ранее – сильный рост ускорения при MPI-параллелизации, небольшое ускорение от добавления OpenMP части и еще один сильный скачок при распараллеливании с помощью CUDA. На Рис. 2, 3, 4 также приведены ускорения для OpenMP, MPI и MPI+OpenMP версий программы.

В таблице 2 приведены времена исполнения разных частей программы. Видно, что основным ”бутылочным горлышком” программы является пересылка данных с GPU на хост и обратно. Это можно исправить путем реализации более оптимальной пересылки данных. В остальном, чистое время вычислений является достаточно небольшим, что говорит об эффективности реализованного подхода.

Таблица 2: Время выполнения различных частей MPI+CUDA программы.

Операция	Время (сек)
Общее время выполнения	1.756
Время инициализации	0.052
Время первого цикла ($n = 0, 1$)	0.083
Время копирования с GPU на хост	0.779
Время копирования с хоста на GPU	0.451
Время обмена	0.051
Время основных вычислений	0.197
Другие операции	0.143

Таблица 3: Результаты расчетов для OpenMP программы.

Число OpenMP нитей	Время решения t , сек	Ускорение S	Погрешность ψ
1	128.346	1.000	4.94e-7
2	64.188	2.000	4.94e-7
4	33.451	3.837	4.94e-7
8	17.967	7.143	4.94e-7
16	10.645	12.057	4.94e-7
32	9.561	13.424	4.94e-7

Таблица 4: Результаты расчетов для MPI программы

Число MPI процессов	Время решения t , сек	Ускорение S	Погрешность ψ
1	122.557	1.000	4.94e-7
4	33.265	3.683	4.94e-7
8	18.220	6.722	4.94e-7
16	9.516	12.878	4.94e-7
32	5.484	22.346	4.94e-7

Таблица 5: Результаты расчетов для MPI + OpenMP программы.

Число MPI процессов	# нитей	Время решения t , сек	Ускорение S	Погрешность ψ
1	1	123.409	1.000	4.94e-7
8	1	23.713	5.204	4.94e-7
8	2	12.220	10.099	4.94e-7
8	4	8.760	14.087	4.94e-7
8	8	5.956	20.722	4.94e-7

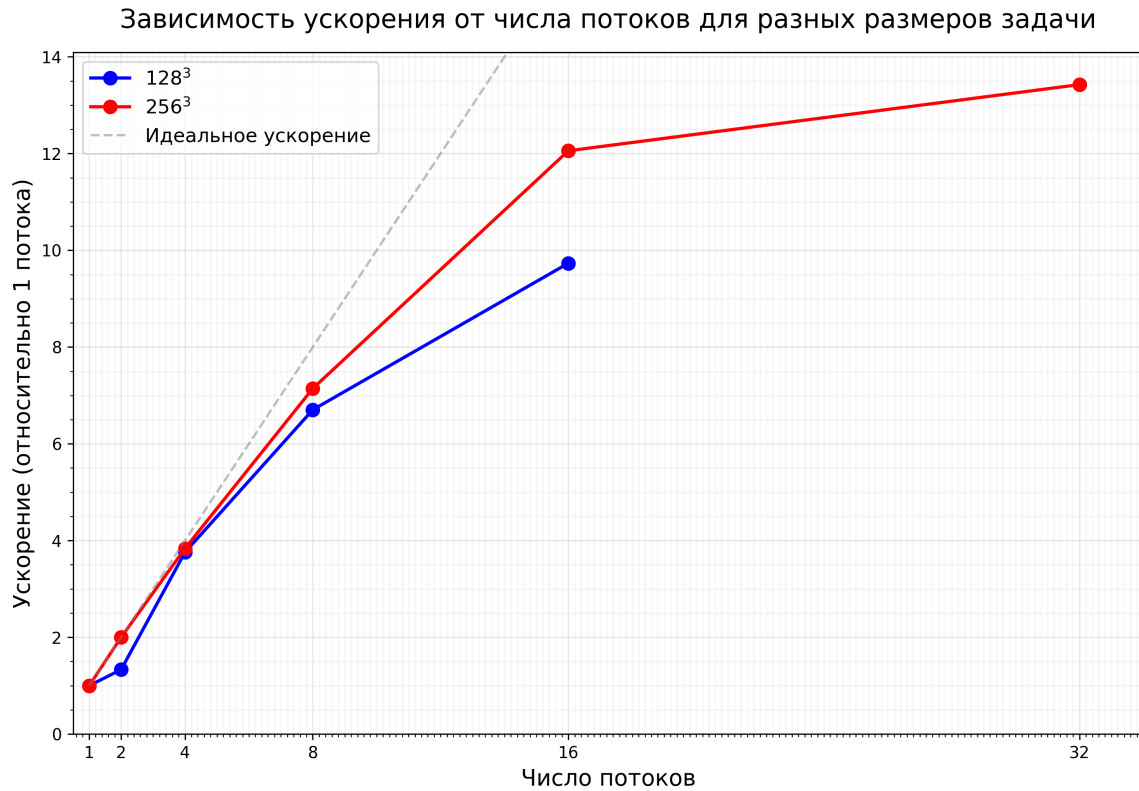


Рис. 2: Графики ускорений для MPI-версии программы при различном числе потоков.

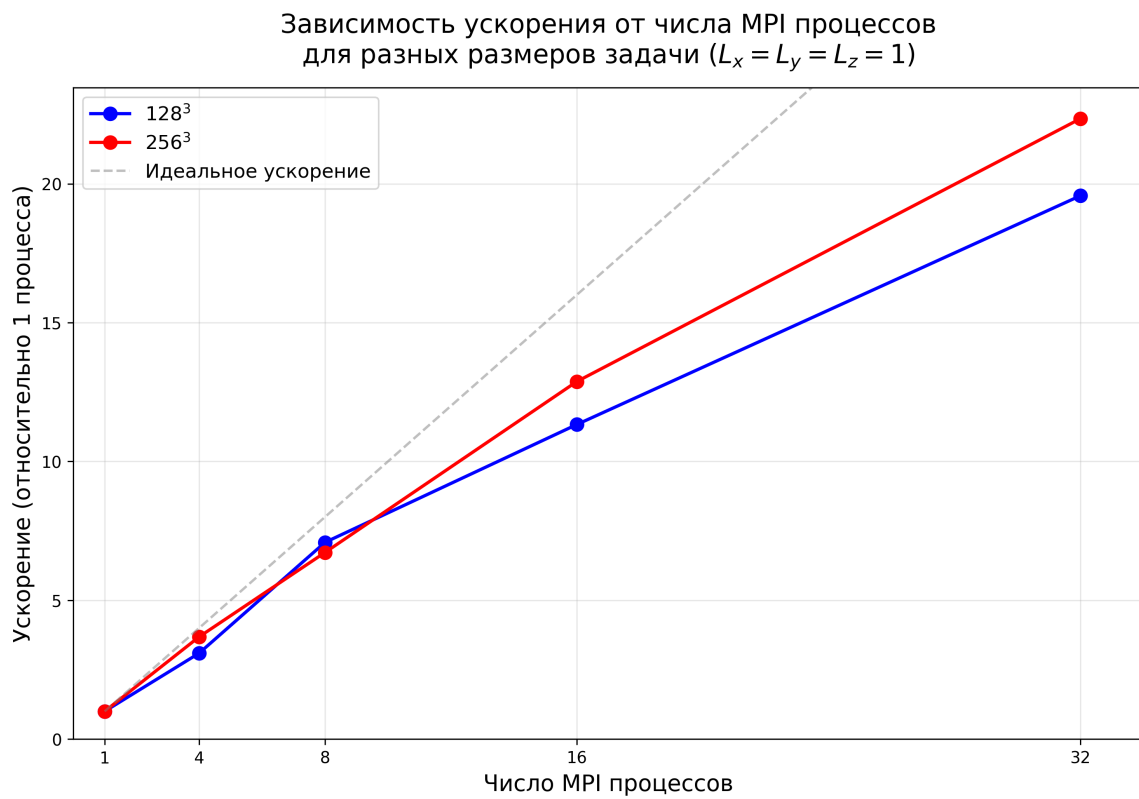


Рис. 3: Графики ускорений для OpenMP-версии программы при различном числе процессов.

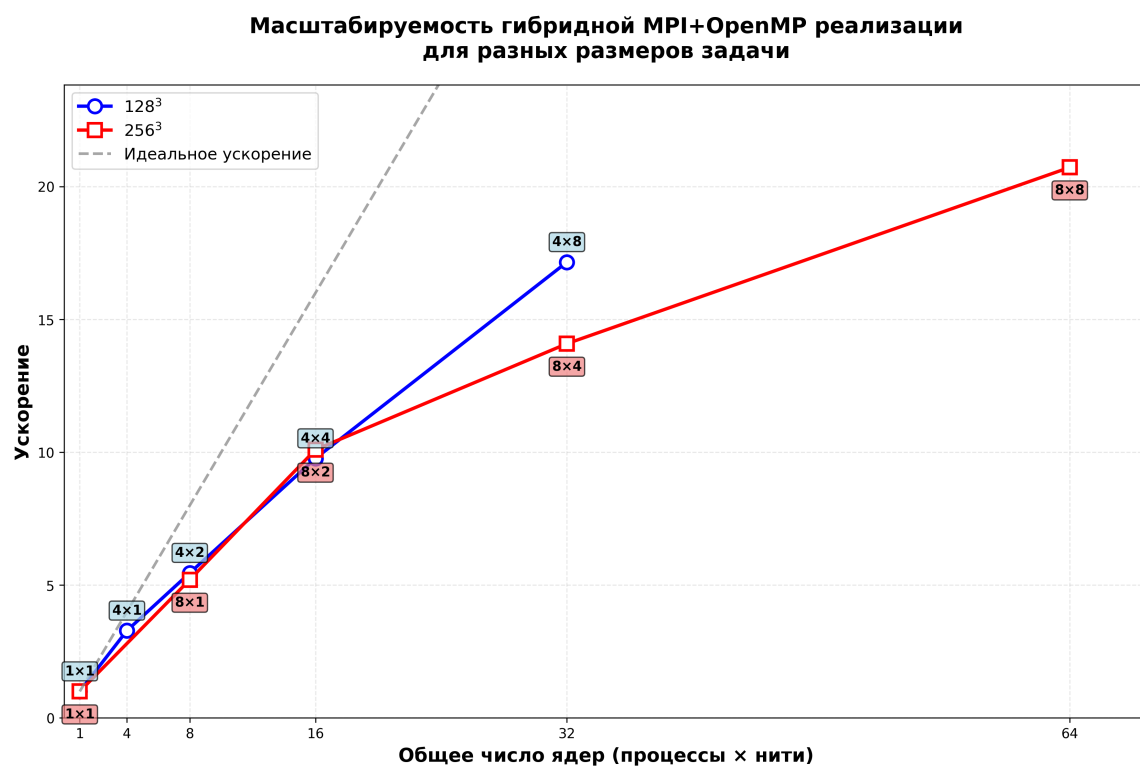


Рис. 4: Графики ускорений для MPI+OpenMP-версии программы при различном числе потоков и процессов.